Guide to Agentic Systems and AI Agents

Learn what agentic systems are, how AI agents work, and how agentic AI automates complex, multi-step workflows across enterprise use cases.

By Databricks

Agentic AI is a class of artificial intelligence in which software systems autonomously plan, execute, and adapt multi-step workflows to achieve specific goals — with minimal human intervention at each step. Where conventional AI tools wait for a prompt and return a single response, agentic systems operate as persistent actors: they perceive context, reason over objectives, call external tools, and refine their behavior based on outcomes.

A traditional AI model receives an input and produces an output; an agentic AI system receives a goal and pursues it across multiple steps, tools, and decisions until the objective is met or a human operator intervenes. This distinction — between responding and acting — is what makes agentic AI a fundamentally advanced form of artificial intelligence and a distinct category from generative AI or traditional machine learning systems.

Choosing between agentic AI, generative AI, and traditional AI models is now a core decision in enterprise AI strategy. The sections below define the key terms, trace how AI agents work, and map the use cases where agentic systems deliver the greatest business value — including agentic analytics, agentic automation, and operational management.

Key Terms: AI Agent, AI System, and Agentic AI System

An AI agent is a goal-directed software entity that perceives its environment through inputs — text, data streams, API responses, sensor feeds — and takes actions to achieve a defined objective. Unlike a static model that maps inputs to outputs, an AI agent maintains state across interactions, decides which large language models or external tools to invoke, and adjusts its approach based on feedback from previous actions.

An AI system is the broader integrated architecture in which agents and models operate. It encompasses the models themselves, the data infrastructure that feeds them, the APIs they call, the memory components that persist information between steps, and the governance layer that controls what the system is permitted to do.

An agentic AI system is an autonomous, goal-driven platform that combines one or more AI agents with the infrastructure required to let those agents operate independently. Agentic AI systems automate complex tasks that would otherwise require sustained human attention — routing decisions, querying multiple data sources, coordinating handoffs between specialized agents. The defining characteristic is autonomous decision making: the system determines how to reach a goal without requiring constant human oversight at each intermediate step.

How AI Agents Work and Agentic AI Workflows

The Perceive-Reason-Act-Learn Loop

AI agents work by cycling through four stages continuously. The agent perceives its environment, ingesting inputs from APIs, databases, user queries, or real-time data streams. It then reasons over those inputs using an LLM or planning module to determine the best next action. It acts by calling a tool, writing to a system, generating content, or

delegating to another agent. Finally, it reflects on the outcome, updating its understanding of task state and feeding that learning into the next perception cycle. This loop runs until the goal is reached or a human operator takes control.

LLMs as the Agent's Reasoning Core

Large language models serve as the cognitive engine of most modern agentic AI systems. The LLM interprets the goal, parses context retrieved from memory and tools, generates a plan of action, and produces the structured outputs — function calls, API parameters, generated text — that drive downstream steps. The most advanced AI systems combine fine-tuned domain models with general-purpose LLMs to balance breadth and precision across different task types. AI agents learn from their experiences when outcomes are written back to long-term memory, allowing agentic AI to improve performance on recurring task types.

Multi-Step Planning and Tool Integration

Agentic AI's ability to autonomously execute multi-step tasks is what distinguishes it from single-turn AI interactions. A complex workflow — investigating a flagged transaction, for example — might require the agent to pull transaction history, cross-reference a sanctions list, calculate risk scores, and route a case to the appropriate reviewer. Agentic systems chain these steps by treating each action's result as the context for the next decision, enabling long-running agents to complete workflows that generative AI models cannot address in a single pass.

Execution depends entirely on external tools — web search APIs, database query engines, code interpreters, communication platforms, and any external system that exposes a programmatic interface. The Model Context Protocol (MCP) is an emerging open standard that specifies how AI agents describe and invoke external tools, enabling interoperability between agents built on different platforms.

Components of Agentic AI Systems and AI Systems Architecture

Perception Inputs and Memory

The perception layer is what makes an agentic AI system situationally aware. Inputs arrive from structured sources like relational databases, semi-structured sources like JSON API responses, unstructured sources like documents and emails, and streaming sources like event queues and sensor feeds.

Memory is what allows agentic systems to operate beyond a single context window. Short-term memory holds the active task context; long-term memory stores user preferences, workflow histories, and domain-specific knowledge retrieved from vector databases. Agentic systems employ external tools to search and monitor data in real time, combining live retrieval with persistent memory to reason over both current conditions and historical context.

Reasoning, Execution, and Orchestration Layers

The reasoning layer is where the agent interprets inputs and decides what to do next. Most production agentic AI systems anchor this layer in one or more LLMs, sometimes paired with specialized planners that decompose high-level goals into subtasks or machine learning algorithms trained on historical workflow data to improve decision quality over time.

The execution layer is where agent decisions become real-world effects — writes to databases, calls to external systems, messages sent through communication platforms, or actions taken in enterprise systems. The orchestration layer coordinates multiple agents and manages the overall workflow: routing tasks to appropriate specialized agents, handling retries when a step fails, managing queues for long-running agents, and providing the observability surface that lets human operators monitor what the system is doing.

Agent Orchestration and Agentic System Coordination

Agent orchestration is the coordination layer that assigns goals to individual agents, sequences their activities, resolves dependencies between tasks, and manages the data flow between agents working in parallel or series. In multi-agent systems — where multiple specialized agents collaborate to complete complex workflows — orchestration prevents agents from producing redundant or conflicting work and ensures that the outputs of one agent become clean inputs to the next.

Multi-agent systems are typically organized in one of two patterns. Hierarchical orchestration uses a supervisor agent that plans the overall task and delegates to specialized worker agents — this works well for stable, well-understood workflows. Decentralized orchestration allows agents to communicate peer-to-peer and self-organize around shared goals, a pattern that is more resilient but harder to audit. Many enterprise deployments combine both patterns within a single agentic system.

Production-grade orchestration requires automatic retry logic for transient failures, task queuing for high-volume workflows, and comprehensive observability — traces, logs, and metrics that show exactly what each agent did and why. Human-in-the-loop escalation paths, where the orchestration layer pauses execution and routes a decision to a human operator, are required for any agentic system taking consequential actions.

Agentic and Generative AI: Differences and Synergies

Generative AI and agentic AI are related but distinct. Generative AI refers to AI models that produce content — text, code, images — in response to a prompt, without executing those outputs against external systems. An LLM asked to draft a vendor evaluation report will produce that draft, but it will not retrieve vendor data, cross-reference contract terms, or route the report for approval. Generative AI produces outputs without executing actions.

Agentic AI uses generative outputs to achieve specific goals. In an agentic system, an LLM's output might be a function call to a database API, a decision to escalate a task, or a structured message to another agent — outputs that cause things to happen in the world. Unlike generative AI, which responds to inputs, agentic AI makes autonomous decisions about what inputs to seek, what actions to take, and how to sequence those actions toward a goal.

The practical guidance for enterprise AI strategy: use generative AI for content creation, summarization, classification, or answering user queries from a fixed context window. Deploy agentic AI when the goal requires multi-step execution, real-time data retrieval, interaction with external systems, or autonomous action affecting other software systems. Mature enterprise deployments use generative models as a component within agentic systems — the LLM reasons; the agent acts.

Autonomous AI, AgentOps, and Governance for Agentic Systems

AgentOps is the operational discipline for managing agentic AI systems in production. An AgentOps practice establishes standards for deploying, monitoring, versioning, and retiring agents, and instruments agents with telemetry that captures decision traces, tool call latencies, error rates, and goal completion rates — the visibility needed to diagnose failures in complex workflows.

Autonomous AI systems require governance controls more granular than those applied to traditional AI. Each agent must carry a distinct identity with a minimal permission set, authorized to access only the data and tools required for its specific function. Policy enforcement should be declarative and auditable, not embedded in agent logic where it can be modified or overridden. An effective AI governance strategy establishes these controls before agents reach production. Sandboxing restricts agents from taking irreversible actions — deleting records, initiating financial transactions — without an explicit human approval checkpoint.

Every action taken by an autonomous agent must be logged with sufficient context to reconstruct the decision that produced it. Full audit trails are required for regulatory compliance and incident investigation. Every agentic workflow should also have a named human owner responsible for outcomes — autonomous agents may act unpredictably if not monitored, and clear human accountability is the governance control that makes autonomous decision making safe enough to deploy at scale.

Risks and Limitations of Agentic AI Systems

Agentic AI can exploit poorly designed reward systems. When a goal is underspecified or when success metrics can be gamed without achieving the intended outcome, agents will find the shortcut: a customer service agent rewarded purely for closing tickets will close them without resolving the issue. Careful goal specification, combined with automated

checks that verify outcomes against intended business results rather than just task completion, is the primary mitigation.

Data privacy concerns arise from agentic AI's data handling — agents that retrieve broad data sets to answer narrow questions process more sensitive information than necessary. Restricting permissions to the minimum required for each agent's function significantly reduces the blast radius of failures. Agentic AI requires strong safety and privacy guardrails embedded at both the agent level and the orchestration layer, not as afterthoughts applied after deployment.

Explainability gaps grow as agentic systems handle more complex decision paths. Organizations should build checkpoints that require agents to generate human-readable rationales for high-stakes decisions and automatically flag low-confidence outcomes for human review before action is taken.

Use Cases: AI Agents in Business and Agentic Systems Examples

Customer Service with AI Agents

Autonomous agents provide 24-hour customer service support, handling routine inquiries — order status, account updates, policy questions — without human involvement. In a well-designed agentic customer service workflow, the agent perceives the incoming request, retrieves the customer's account data from CRM and support systems, takes a direct action or generates a response, and closes the ticket — all without a human agent involved.

Exceptions are automatically escalated to human teams, ensuring that autonomous AI handles repetitive tasks while human agents focus on complex, relationship-sensitive interactions. Agentic AI's ability to automate high volumes of repetitive tasks while maintaining consistent quality is among its clearest enterprise value propositions.

Software Development and AI Agent Assistance

In software development workflows, agentic AI systems generate code, run unit tests automatically, open pull requests, and annotate changes for human reviewers. An agent assigned to a bug report can reproduce the issue in a sandboxed environment, identify the offending code path, generate a fix, run the relevant test suite, and prepare a pull request — all before a human engineer reviews the work.

This compresses time-consuming tasks in the development lifecycle and lets engineering teams focus on architecture and review rather than routine implementation work. Agentic AI's ability to operate across the full software development workflow, from issue triage to code submission, is one of the most active areas of enterprise automation investment.

Supply Chain and Logistics Agentic Systems

Agentic AI can autonomously manage supply chain operations end to end. Supply chain management systems powered by agentic AI monitor inventory in real time, predict demand fluctuations, and place replenishment orders automatically when stock falls below dynamically adjusted thresholds.

More advanced deployments use API-driven agent transactions to optimize supply chains further — querying pricing systems, comparing vendor lead times, and selecting suppliers within predefined rules. Agentic AI can optimize inventory levels based on demand fluctuations without human supervision, enabling supply chain teams to manage by exception rather than constant monitoring.

Finance and Risk Management with Agentic Systems

Agentic systems manage high-speed analysis for compliance and fraud detection at a scale and speed that human teams cannot match. A financial services firm might deploy an agent that continuously monitors transaction streams for anomalous patterns — flagging suspicious activity, retrieving account history, scoring risk, and initiating an investigation workflow automatically.

AI trading bots analyze market data to execute trades autonomously within predefined risk parameters. Separately, reporting agents synthesize regulatory data and generate draft submissions for human review before external filing. In each case, agentic AI handles the high-volume, time-consuming tasks while human teams retain authority over consequential decisions.

Building and Deploying an AI Agent and Agentic AI System

The first decision in building a production-ready agentic system is framework selection: teams can build in Python using low-level orchestration primitives, adopt an open-source agent framework, or build on a managed enterprise platform like Agent Bricks with pre-built components for common agent patterns. Managed platforms reduce time to deployment; custom builds offer more control but require deeper engineering investment.

The recommended starting point is a minimal agent with sandboxed tool access — a single agent with a narrow goal, access only to the tools it needs, and an environment where its actions cannot affect production systems. This prototype validates the core perceive-reason-act loop before complexity is added. Securing APIs and managing credentials centrally is non-negotiable at this stage. Agents that handle credentials insecurely create exploitable attack surfaces; a centralized secrets management system with per-agent scoping and automated rotation is the correct pattern.

Every agentic system should emit structured telemetry — traces for each agent decision, metrics for tool call success rates and latencies, and failure alerts for human operators. Instrumentation is the foundation of AgentOps practice and what enables teams to build the operational confidence that justifies expanding agent autonomy over time.

Measuring ROI and Operating Agentic AI Work in Production

Defining success metrics tied to business outcomes — not just technical metrics — is the first step in measuring agentic AI value. An agentic customer service system should be evaluated on customer satisfaction and resolution rates, not just ticket volume. An agentic supply chain system should be measured on inventory turnover and procurement cost, not just automated purchase orders generated.

Alongside business metrics, track operational metrics for each agent: workflow success rate, tool call failure rate, average cost per workflow, and time to completion relative to the human baseline. These metrics feed continuous validation cycles that identify underperforming agents and surface failure modes before they impact business outcomes. Model versioning is an underappreciated requirement — when the underlying LLM is updated, agent behavior can shift in ways not immediately visible in aggregate metrics. Running regression tests against a representative sample of historical tasks before promoting a new model to production prevents unexpected behavior changes.

Future Trends: Agentic Systems, Autonomous AI, and Agentic AI Work

By 2028, 15% of work decisions will be made autonomously by agentic AI — a projection that signals how quickly autonomous decision making is shifting from experimental to operational.

Interoperability standards are emerging as the infrastructure layer that will determine whether multi-agent systems can operate across organizational and platform boundaries. Model Context Protocol (MCP) and similar protocols specify how AI agents describe and invoke external tools, enabling agentic systems from different vendors to collaborate in shared workflows. As these standards mature, enterprises will compose agentic workflows from specialized agents built by different providers.

Agent marketplaces will accelerate enterprise automation timelines. Rather than building every agent from scratch, organizations will procure specialized AI agents — compliance monitors, vendor negotiation agents, clinical screening agents — from marketplaces of pre-built domain-specific offerings. This shift will require new governance practices for evaluating and auditing third-party agents. As agentic systems take on more autonomous decision making, roles like agent orchestration architect, AgentOps engineer, and AI governance specialist will become standard functions within technology organizations.